

IPVS Load Balancing Techniques

LVS/NAT: This technique is one of the simplest to set up but could present an extra load on the load balancer, because the load balancer needs to rewrite both the request and response packets. The load balancer needs to also act as a default gateway for all the real servers, which does not allow the real servers to be in a geographically different network. The packet flow in this technique is as follows:

- The load balancer examines the destination address and port number on all incoming packets from the client(s) and verifies if they match any of the virtual services being served.
- A real server is selected from the available ones according to the scheduling algorithm and the selected packets are added to the hash tables recording the connections.
- The destination address and port numbers on the packets are rewritten to match that of the real server and the packet is forwarded to the real server.
- After processing the request, the real server passes the packets back to the load balancer, which then rewrites the source address and port of the packets to match that of the real service and sends it back to the client.

LVS/DR: DR stands for Direct Routing. This technique utilises MAC spoofing and demands that at least one of the load balancer's NIC and real server's NIC are in the same IP network segment as well as the same physical segment. In this technique, the virtual IP address is shared by the load balancer as well as all the real servers. Each real server has a loop-back alias interface configured with the virtual IP address. This loop-back alias interface must be NOARP so that it does not respond to any ARP requests for the virtual IP. The port number of incoming packets cannot be remapped, so if the virtual server is configured to listen on port 80, then real servers also need to service on port 80. The packet flow in this technique is as follows:

- The load balancer receives the packet from the client and changes the MAC address of the data frame to one of the selected real servers and retransmits it on the LAN.
- When the real server receives the packet, it realises that this packet is meant for the address on one of its loopback aliased interfaces.
- The real server processes the request and responds directly to the client.

LVS/TUN: This is the most scalable technique. It allows the real servers to be present in different LANs or WANs because the communication happens with the help of the IP tunnelling protocol. The IP tunnelling allows an IP datagram to be encapsulated inside another IP datagram. This allows IP datagrams destined for one IP address to be wrapped and redirected to a different IP address. Each real server must support the IP tunnelling protocol and have one of its tunnel devices configured with the virtual IP. If the real servers are in a different network than the load balancer, then the routers in their network need to be configured to accept outgoing packets with the source address as the virtual IP.

This router reconfiguration needs to be done because the routers are typically configured to drop such packets as part of the anti-spoofing measures. Like the LVS/DR method, the port number of incoming packets cannot be remapped. The packet flow in this technique is as follows:

- The load balancer receives the packet from the client and encapsulates the packet within an IP datagram, and forwards it to a dynamically selected real server.
- The real server receives the packet, 'de-encapsulates' it and finds the inner packet with a destination IP that matches with the virtual IP configured on one of its tunnel devices.
- The real server processes the request and returns the result directly to the user.

2 (where *rproxy2* is located) is set to allow the spoofed address containing the VIP as the source address to pass through. This is the most common problem people face.



Note: If all the real servers are in the same data centre, then you need not worry about router configuration, but in that case you are better off using LVS/DR, which will give you faster performance.

High availability of load balancers—connection synchronisation in IPVS

The *ipvsadm* command can run in daemon mode either as a master or back-up. The master server synchronises the connection information to the back-up server, thus providing a persistent connection in case the master server needs to be failed over to the back-up server.

The high-speed operation of a load balancer can cause some amount of extra load on the load balancers. So we need to make sure that the connection

synchronisation does not become an overhead to the overall operations. According to the linux.virtualsever.org website, at least the following connection information needs to be passed to the back-up from the master, which is around 24 bytes:

```
<Protocol, CIP,CPort, VIP,VPort, RIP,RPort, Flags, State>
```

Efficient synchronisation is done using UDP multicast inside the Linux kernel. The master load balancer runs the IPVS syncmaster daemon inside the kernel, passing the connection information with the UDP multicast to the back-up load balancer(s) accepting the UDP multicast packet.

On the primary load balancers in each data centre, run the following code:

```
ipvs@b11# ipvsadm --start-daemon=master --mcast-interface=eth1
ipvs@b21# ipvsadm --start-daemon=master --mcast-interface=eth1
```